

Web Crawler Part 3: Proof of Concept

1. POC Definition:

The complete, billion-URL system won't be built at once. First, I'll build a small version to prove that the risky parts work before spending time and money at scale.

Goal: Crawl 10 million URLs across ~20 domains in one week, running on the real pipeline design from Part 2 (file -> queue -> fetch workers -> storage -> parse workers -> table).

Rationale: 10 million is big enough to hit the behaviors that matter, like per-domain politeness limits, queue backpressure, machine crashes, and real cost-per-URL numbers. However, it is still small enough that I can rerun the whole thing in a day if something breaks.

Targets In-Scope Now:

- Loading URLs from a file, cleaning them, and removing duplicates
- The queue with per-domain politeness control
- Fetch workers saving raw HTML, running on cheap interruptible machines
- The fetch/parse split, including one full re-parse of stored HTML without re-crawling
- The 35-column schema, end-to-end, into a queryable table
- Basic dashboards and alerts

Targets Out-of-scope for Now:

- Live MySQL ingestion (a monthly file export is enough to prove the concept)
- The giant retailers (Amazon, Walmart). These are blocked by bot defenses and need an API/licensing deal.
- Headless browsers and multi-region setups

2. Implementation Schedule:

Phase	Weeks	Goal	Exit gate
P0: Foundations	1-2	Set up the cloud infrastructure; upgrade fetcher.py to handle many downloads at once; build the URL loader (clean + dedup); lock the 35-column schema	100,000 URLs run end-to-end on one machine
P1: Scaling	3-4	Build the per-domain	1 million URLs

		politeness controller; make fetch workers auto-scale; add the queue with automatic retries; parse workers write into the table	processed with zero manual babysitting
P2: Monitoring	4-5	Build the dashboards and alerts; run a crash test that kills workers mid-run to prove no URLs are lost	Can answer "Is the batch on track?" from a dashboard alone
P3: Evaluation	6	Full 10M-URL run; one re-parse run over stored HTML; write the evaluation report	Go / no-go decision made

- With some padding, the realistic range to complete the POC is 6-10 weeks. P0 is high-confidence because it is mostly updating code that has already been written and tested. P1 is the riskiest phase because the politeness controller is a brand-new component. P2 and P3 are lower-risk.

3. POC Evaluation:

The POC succeeds if it hits clear targets in four areas. Every number goes into a short evaluation report at the end.

- **Does the system perform?**

Metric	Target	Rationale
Sustained Speed	at least 50 URLs/sec for 24 hours straight	proves steady throughput, not a lucky burst
Per-Domain Politeness	zero rate-limit violations	We want to get this right, so we don't get blacklisted
Crash Recovery	zero URLs lost across 5+ machine interruptions	proves the queue design actually protects work

- **Is the data good? (checked on a sample of ~1,000 records per domain, plus a hand-audit of 500)**

Metric	Target
Fetch Success Rate	>= 85% reachable per domain
Metadata Completeness	>= 95% of good fetches have a title
Classification Accuracy	>= 85% correct on hand-audit sample
Re-parse Test	100% of stored HTML re-parsed with zero re-crawling

- **Does the cost make sense?**

Metric	Target
Cost per 1,000 URLs	Within 1.5X of the Part 2 Estimate
Projected cost at 1 billion/month	Within 2X of the model, with any gap explained

- **Do the operations work?**
 - The answer is yes if: Every alert fired at least once (from the crash test or naturally) and each one was actually useful, and the "projected completion date" on the dashboard stayed within 10% of what actually happened.

Decision Criteria:

Go: all performance and data targets hit, cost in range.

Conditional Go: a few misses, but each is explained by a known, fixable cause.

No-Go: the politeness math or block rates make the timeline impossible by crawling, which means escalating to the API/licensing path from Part 2 before spending more on engineering.

4. Blockers and Estimated Arrival Times:

Known and Smaller:

Blocker	How To Resolve	Time to Clear
AWS account limits (max machines, max queue size)	request limit increases early	1–3 days
Upgrading fetcher.py to run many downloads at once introduces bugs	the 29 existing tests catch behavior changes during the upgrade	handled within P0
Getting the URL source in MySQL format	one meeting plus a sample export; the loader reads either format	1 week (calendar, not effort)
Downstream teams signing off on the 35-column schema	share it in week 1; only add columns after, never rename	1 week

Known and Harder:

Blocker	Why it's serious	How I'd handle it	Time to clear
Bot walls on the big retailers	I already measured this. Reddit hard-blocks AWS IPs, Amazon serves fake pages. Plain crawling of Amazon/Walmart may succeed under 20% of the time	run the PoC on reachable mid-size domains; start a parallel track in week 1 to get official APIs or licensed data for the retailers	4–8 weeks, and it's on the critical path for the retail sites
Legal / terms-of-service review	crawling policy and any paid proxy service need legal sign-off before use	engage legal in week 1 with the tier options from Part 2; polite, robots-respecting crawling proceeds meanwhile	2–6 weeks

The politeness-vs-deadline decision	some domains can't be finished in a month by crawling — someone has to decide: crawl slower, pay for more IPs, or license the data	present the math and the cost of each option at the end of P1; the decision becomes part of the SLA, not an engineering guess	one decision meeting, ~week 4
-------------------------------------	--	---	-------------------------------

Unknowns I can't list yet: I've added up to 4 extra weeks of padding in the schedule to account for further unforeseen challenges. Part 1 brought up a few of those, like a Docker bug on my machine to test my deployment, an AWS account tier lockout, and some Windows tooling issues that were never anticipated. The schedule assumes that some bugs, such as these, will inevitably arise.

5. Release Plan and Shipping:

Each stage has to pass the same checks as the POC in roughly 10X steps, before moving up, so "it worked at this size" is always proven, never assumed.

Stage	Scale	Timing	What lets it advance
Proof of Concept	10 million URLs, 20 domains	week 6	passes the Section 3 checks
Beta	100 million URLs, first real monthly batch, MySQL ingestion added	weeks 7–10	targets held for a full batch, cost within 1.5x of the model, no data lost
GA (full launch)	1 billion+/month, SLA signed, on-call rotation live	weeks 11–16	two monthly batches in a row that meet the SLOs

High-Quality Verification:

Roughly 4 months from POC to full production, tested and verified at every step in ~ 10X scale steps.

- Version everything: Every image and data record is tagged, so I always know which code produced which data (already built in via `schema_version` and `classifier_version`).
- Canary releases: A new version runs on 5% of one domain for 24 hours and is compared against the old version before full rollout; if it performs worse, roll back instantly.

- Data changes get before/after checks: an improved classifier is re-run on stored HTML and its accuracy compared, not just shipped.
- Runbooks and on-call: before beta, every alert has a written response guide and an on-call rotation; if reliability slips too far, new features pause until it's fixed.
- Deferred to after launch: scheduled re-crawling, headless browsers for the retailer track, and multi-language topics. These are real improvements, but not needed to prove the system or run the first batches.